주차 진행 보고

LLM 기반 DAST 하네스

SQL Injection · XSS 자동 진단 파이프라인 — 진행 상황

크롤링 › 도구 선택 › 스캔 › 판단 › 리포트

2026-10 · 작성자: 박근령

01 프로젝트 개요

웹 애플리케이션의 SQL Injection · XSS 취약점을 LLM 판단 기반으로 자동 진단하는 경량 DAST 하네스

목표

SQLi·XSS 자동 진단 + 오탐 최소화

방식

크롤링→선택→스캔→판단→리포트 5단계

범위

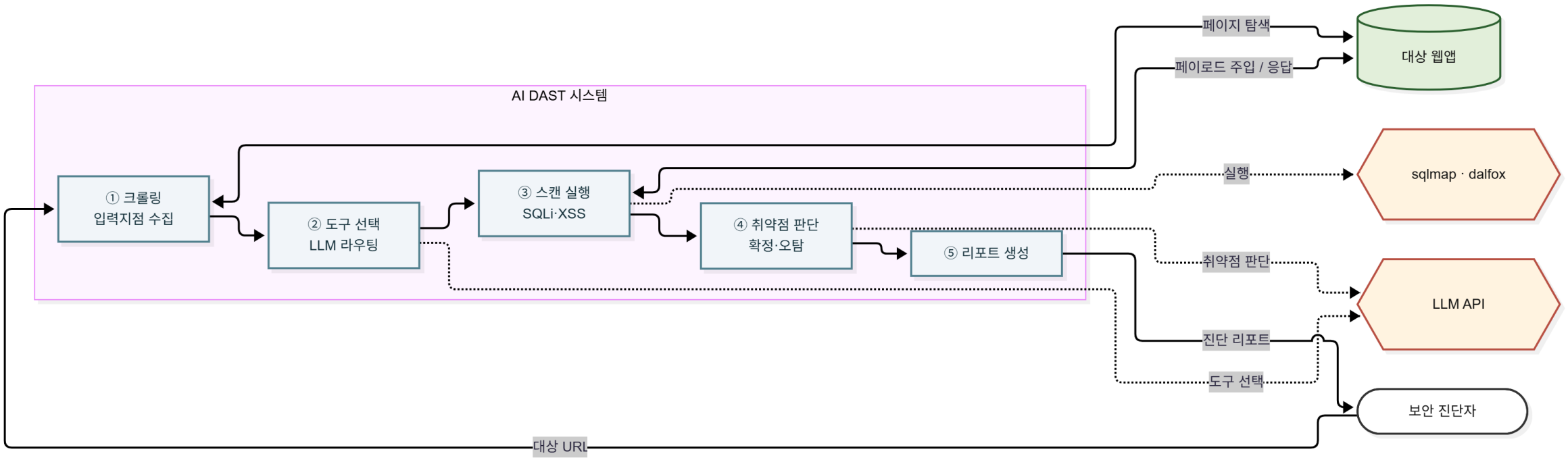
SQLi·XSS 2종 (확장 가능 구조)

안전

비파괴·속도제한·범위제한 (NFR) 강제

핵심 아이디어: 강력한 스캐너(sqlmap·dalfox)에 LLM '판단 계층'을 결합 → 도구 선택·오탐 제거를 자동화

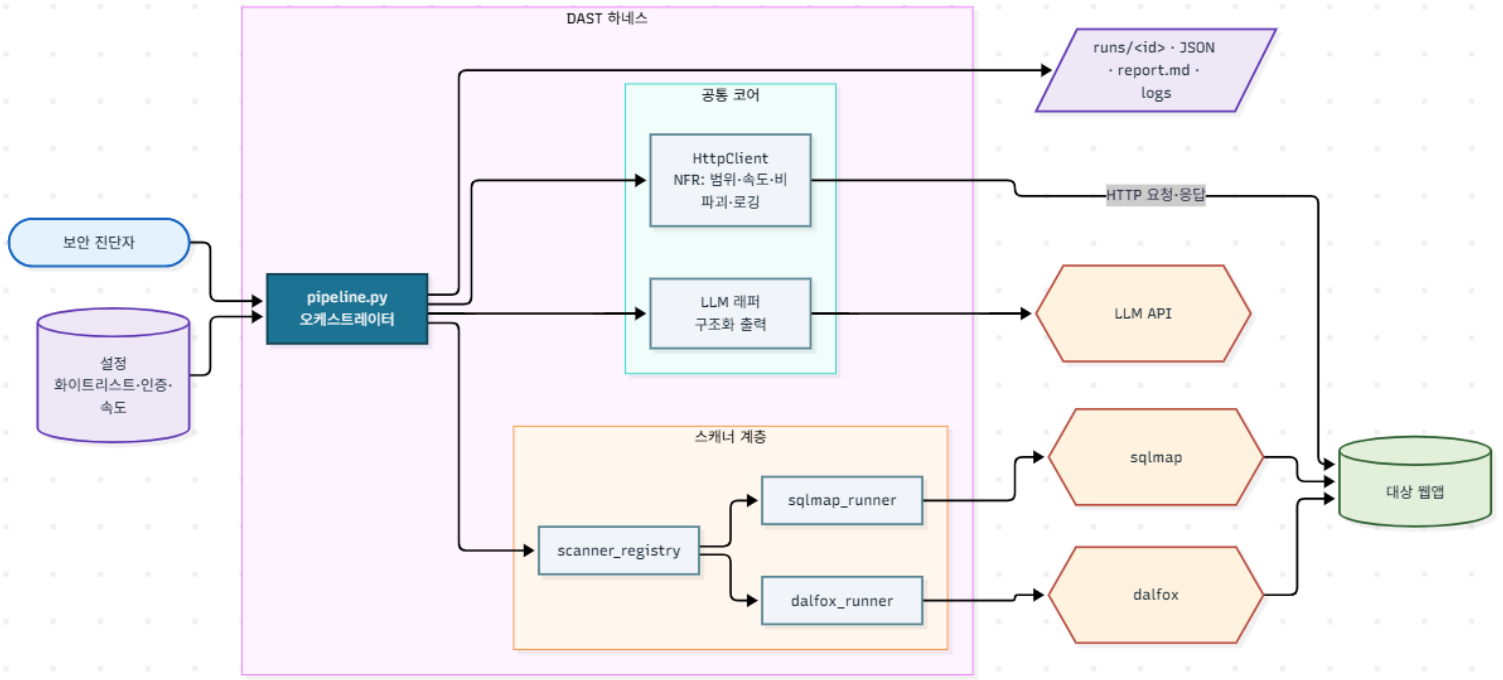
02 시스템 설계 구성도 (한눈에)



사용자 → ① 크롤링 → ② 도구 선택 → ③ 스캔 → ④ 판단 → ⑤ 리포트 (외부: sqlmap · dalfox · LLM API)

핵심: 모든 HTTP 요청은 NFR 공통 레이어(범위·속도·비파괴·원문로깅)를 통과한다.

02 시스템 설계 구성도 (상세 아키텍처)



구성 요소

- **pipeline.py**
  - 5단계 오케스트레이터
- **core/**
  - HttpClient(NFR)·LLM래퍼·schemas
- **scanners/**
  - registry·sqlmap·dalfox runner
- **tools/**
  - sqlmap · dalfox (외부 바이너리)

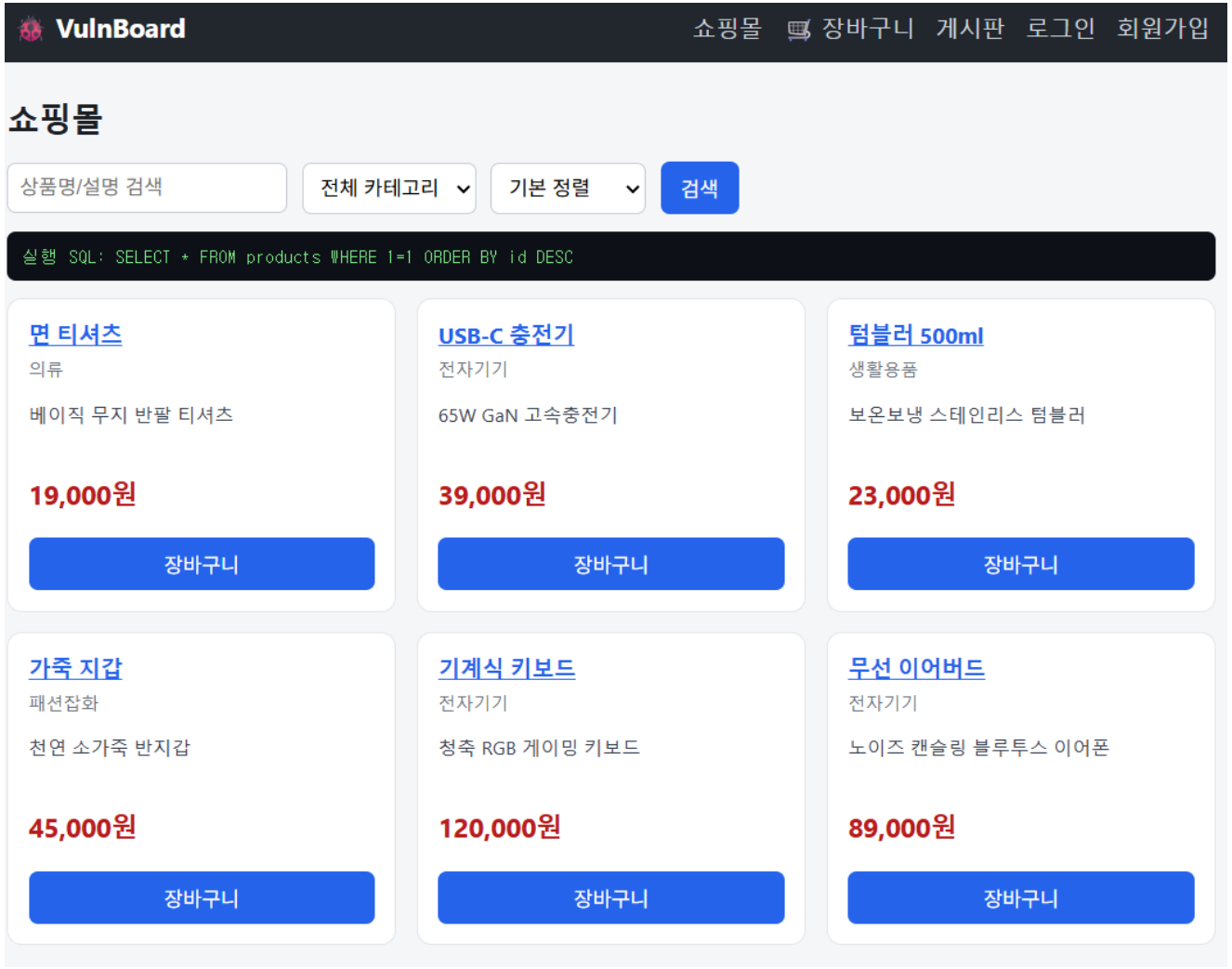
03 진행 ① 크롤링 (FR-01) — 원리

- **시작 URL에서 링크(<a href>)를 따라가며 내부 페이지를 자동 탐색**
- 외부 도메인 링크는 제외 · 탐색 깊이/페이지 수 상한 적용
- **각 페이지에서 입력지점 추출:**
  - GET 쿼리 파라미터 (?q=, ?id=)
  - 폼 필드 (input / textarea / select → POST는 body)
- **수집 결과를 01_crawl.json 으로 저장 → 다음 단계 입력**

시작 URL에서 링크을 열고 들어가며 '입력 양식(폼·검색창)'을 적어둔 후 . 이미 가본 페이지는 다시 안 가고,
너무 깊이·너무 많이는 안 간다.

03 진행 ① 크롤링 — 실측 결과

로컬 취약 앱 'flask+Mysql을 활용한 취약한 웹 시스템' 크롤링




```
  "id": "asset-003",
  "url": "http://127.0.0.1:8087/board",
  "method": "GET",
  "status": 0,
  "content_type": "",
  "raw_response_path": "",
  "injection_points": [
    {
      "id": "ip-016",
      "location": "query",
      "param": "field",
      "param_type": "string",
      "sample_value": ""
    },
    {
      "id": "ip-017",
      "location": "query",
      "param": "q",
      "param_type": "string",
      "sample_value": ""
    }
  ]
},
```

03 진행 ① 크롤링 — 실측 결과

로컬 취약 앱 'flask+Mysql을 활용한 취약한 웹 시스템' 크롤링 결과

31	42	25
방문 페이지	입력지점(파라미터)	자산(엔드포인트)

추출된 입력지점 예시

ID	파라미터	위치	기능
ip-001	q	query	검색 (shop)
ip-018	username	body	로그인
ip-004	product_id	body	장바구니
ip-054	title	body	게시글 작성
ip-058	content	body	댓글

03 진행 ① 크롤링 — 안전장치 실증

✓ 비파괴 가드(NFR-01) 적용

크롤링 중 게시글 삭제 링크(/board/N/delete)를 따라가려 하자, HttpClient의 비파괴 필터가 'delete' 패턴을 감지해 요청을 자동 차단. → 설계한 안전 레이어가 실제로 '게시글 삭제' 사고를 예방.

```
GET /board/7/delete  →  ⚠️ 차단: 비파괴 정책 위반 패턴 감지 'delete'
GET /board/6/delete  →  ⚠️ 차단 …  (7건 모두 차단)
```

의미: '효율'보다 '안전'을 우선하는 설계 — 크롤러가 상태를 바꾸는 링크는 아예 건드리지 않는다.

04 진행 ② 판단 (FR-02 도구 선택) — 방식

- **입력지점의 메타데이터(이름·타입·위치)를 보고 어떤 스캐너(SQLi/XSS)를 적용할지 판단**
- LLM 출력은 구조화(JSON)로 강제 → 프로그램이 파싱 (자연어 파싱 지양)
- 판단 근거(reasoning)·신뢰도를 함께 기록 → 추적·재현
- 02_select.json 으로 저장 → FR-03 스캔이 이 결정대로 실행

판단 예시

- q (검색어) → DB 조회 + 결과 반사 → SQLi + XSS
- sort → ORDER BY 정렬 쿼리 → SQLi 최우선
- product_id (hidden) → DB 식별자, 반사 안됨 → SQLi만

04 진행 ② 판단 (FR-02 도구 선택) — 방식

ip-001 · `q` (query, string)

- 대상: `GET http://127.0.0.1:8087/shop`
- **선택**: ['dalfox', 'sqlmap'] / **우선순위**: dalfox → sqlmap
- **Gemini 판단 근거**: 검색어 파라미터 'q'는 응답에 반영되어 XSS(Reflected) 취약점 발생 가능성이 높으며, DB 조회에 사용될 수 있어 SQLi 가능성도 함께 존재합니다.
- **신뢰도**: 0.9
- **이 판단을 공유하는 입력지점(2개)**: ip-001, ip-017

ip-002 · `category` (query, string)

- 대상: `GET http://127.0.0.1:8087/shop`
- **선택**: ['sqlmap', 'dalfox'] / **우선순위**: sqlmap → dalfox
- **Gemini 판단 근거**: 대상 파라미터 'category'는 데이터베이스 조회(SQLi)와 응답에 반영(XSS)될 가능성이 모두 존재하므로 두 스캐너를 모두 선정했습니다.
- **신뢰도**: 0.85

04 진행 ② 판단 — 규칙 vs 실제 LLM

규칙 기반 (MockLLM)

- 이름에 id/num·숫자형 → SQLi 우선
- 그 외 문자열 → XSS 우선
- 맥락 모름 → 42개 전부 '둘 다, XSS 우선'
- 장점: 빠름·무료·결정적
- 단점: 맥락 못 읽음

실제 LLM (임시로 Gemini)

- 맥락을 읽어 판단 (sort→ORDER BY 등)
- 필요 없으면 한 종류만 선택 → 스캔 수 ↓
- 판단 근거를 문장으로 설명
- 장점: 맥락 이해·정교
- 단점: 느림·할당량·흔들림(비결정성)

→ 실제 LLM이 규칙보다 맥락을 잘 읽지만, '흔들림'과 '미탐'이라는 새 과제를 동반 (다음 장)

05 문제점 (보안 관점)

보안 진단에서 가장 중요한 두 가지: ① 일관성(재현성) ② 보수적 탐지(놓치지 않기)

비결정성

같은 입력이라도 호출/모델마다 판단이 달라진다. →
같은 진단을 다시 돌리면 결과가 바뀔 수 있음
(재현성 저하)

미탐 (False Negative)

치명적인 취약 지점인데 '안전'으로 판단되어 통과하는 경우가 존재한다. →
진단에서 가장 위험한 실패 (있는데 못 찾음)

ip-003 · `sort` (query, string)

- 대상: `GET http://127.0.0.1:8087/shop`
- **선택**: [] / **우선순위**: (없음)
- **Gemini 판단 근거**: 정렬(sort) 파라미터는 순수 표시 및 제어용 문자열로, 데이터베이스 조회나 응답 반사로 인한 취약점 발생 가능성이 낮아 스캔을 생략합니다.
- **신뢰도**: 0.95

05 문제점 — 실제 미탐 사례

동일한 `sort` 파라미터, 모델에 따라 판단이 갈림 -> 추후 Claude api 변경

큰 모델 (flash)

"sort는 ORDER BY 정렬 쿼리에 쓰여 SQL Injection 최우선"
->그러나 같은 모델이어도 가끔씩은 서로 다른 답변

✓ 올바른 판단 (ORDER BY는 유명한 SQLi 벡터)

가벼운 모델 (flash-lite)

"sort는 순수 표시/제어용 문자열 → 취약 가능성 낮아 스캔 생략" (신뢰도 0.95)

✗ 미탐 — 실제 취약점을 '안전'으로 쳐냄

왜 위험한가

FR-02(판단)에서 skip하면 FR-03(스캔)이 그 파라미터를 아예 보지 않는다. →
실제 취약점이 있어도 영구 미탐. 게다가 LLM은 0.95의 높은 신뢰도로 '안전'이라 답함 → 신뢰도 ≠ 정답.

06 해결 방안

① 일관성 ↑

- **temperature = 0 — 출력 흔들림 최소화**
- 모델 버전 고정
- 결정 캐싱 — 같은 입력은 저장된 판단 재사용 → 완전 재현
- 프롬프트 규칙·예시 강화

② 미탐 ↓ (보수적 탐지)

- **안전 가드레일 — 위험 파라미터 skip 금지**
- 의심되면 둘 다 스캔 (놓치는 것보다 과검이 안전)
- LLM + 규칙 하이브리드 — LLM은 우선순위만, 제외는 규칙이 막음
- FR-04 재현으로 재확인

설계 원칙: "LLM은 우선순위만 정하고, 무엇을 쳐낼지(제외)는 규칙이 막는다" — 변동성·미탐을 구조로 방어

차주 계획

- 일관성·미탐 대응 적용 — temperature=0 + 안전 가드레일(skip 금지/보수적 둘 다 스캔)
- 실제 Claude (API) 연동
- FR-03 스캔(sqlmap·dalfox) → FR-04 판단 → FR-05 리포트까지 연결
- 결정 캐싱으로 재현성 확보

요약: 1~2단계(크롤링·판단) 동작 확인 완료 · 보안 관점 한계(일관성·미탐) 식별 · 차주 보완 예정